

Esame OS Internals (Cabodi)

Data: 10/07/2020

Nota Generale: TUTTE LE RISPOSTE SI/NO DEVONO ESSERE MOTIVATE. QUANDO I RISULTATI SONO NUMERI, SONO RICHIESTI SIA IL RISULTATO FINALE SIA I RELATIVI PASSI INTERMEDI (O FORMULE).

1 SECTION 1: THEORETICAL (Silberschatz)

Domanda 1 (Memoria Virtuale, Working Set e Codice C)

Contesto: Sia dato il frammento di programma rappresentato:

```
#define N 512
float M[2*N][N], V[2*N*N];
int i, j, k, t;

for (i=t=0; i<2*N; i++) {
    k = (i<N) ? N-i : i-N+1;
    for (j=0; j<k; j++) {
        V[t++] = M[i][j];
    }
}
```

Il codice macchina generato da tali istruzioni viene eseguito in un sistema con gestione della memoria basata su demand paging (paginazione a richiesta), pagine da 2 KByte, utilizzando come politica di sostituzione pagine la "working set" (versione esatta) con $\delta = 10$. Si sa che:

- la dimensione di un `float` è di 32 bit;
- il segmento di codice (istruzioni in codice macchina) ha dimensioni inferiori a una pagina;
- `M` e `V` sono allocati ad indirizzi logici contigui, a partire dall'indirizzo logico `0x74240A00`;
- la matrice `M` è allocata seguendo la strategia "row major".

Si risponda alle seguenti domande:

- A) Quante pagine (e frame) sono necessarie per contenere la matrice `M` e il vettore `V`?
- A) Ipotizzando che le variabili `i`, `j`, `k`, `t` siano allocate in registri della CPU, quanti accessi in memoria (in lettura e scrittura) fa il programma proposto per accedere a dati (non vanno conteggiati gli accessi a istruzioni)?
- A) Calcolare il numero di page fault generati dal programma proposto, supponendo che siano allocati per esso 10 frame, di cui uno utilizzato (già all'inizio dell'esecuzione) per le istruzioni. (Considerare caso minimo e massimo, e motivare le risposte).

Domanda 2 (RAID e Affidabilità)

Contesto: Sia dato un disco organizzato con struttura RAID. Sono dati il tempo medio tra guasti (MTTF), il tempo medio per riparazione (MTTR), e il tempo medio tra perdite di dato (MTTDL).

Si risponda alle seguenti domande:

- A) E' possibile che $MTTR > MTTF$?
 - A) E' possibile che $MTTDL > 10 \cdot MTTF$?
 - A) Si consideri una struttura RAID con 2 dischi in configurazione "mirrored". Se ognuno dei dischi può guastarsi in modo indipendente dall'altro, con $MTTF = 20000$ ore, quanto deve essere MTTR per garantire $MTTDL > 100 \cdot MTTF$?
-

Domanda 3 (I/O Sincrono, Asincrono e Polling)

Si risponda alle seguenti domande:

- A) Si spieghino le principali differenze tra gli I/O dei due tipi (sincrono e asincrono). Si considerino poi I/O bloccante e non bloccante: si tratta di sinonimi di asincrono e sincrono? Ci sono differenze? (se no, perché; se sì, quali?).
- A) Un I/O sincrono può essere effettuato in polling oppure è necessario effettuarlo in interrupt? (Si risponda alla stessa domanda per il caso dell'I/O asincrono).
- A) In quale modo può un processo beneficiare di un I/O asincrono? E' possibile, nel caso di I/O asincrono, scrivere un programma con istruzioni, successive all'I/O, che dipendano dai dati coinvolti (ad es. letti) durante l'I/O?

2 SECTION 2: OS161

Domanda 4 (System Call `sys_read` e Sicurezza)

Contesto: Sia dato un sistema OS161. Si consideri la funzione `sys_read`, utilizzata per realizzare la system call `read`. Se ne propone una realizzazione, limitata al caso di standard input:

```
int sys_read(int fd, userptr_t buf, int size) {
    if (fd == STDIN_FILENO) {
        kgets ((char *)buf, size);
        return size;
    } else {
        kprintf("sys_read implemented only on stdio\n");
        return -1;
    }
}
```

Si risponda alle seguenti domande:

- B) E' possibile che la funzione sia richiamata direttamente da un processo user? (Motivare).
 - B) La funzione è errata, in quanto la chiamata a `kgets`, così come effettuata, rischia di fare un accesso scorretto in memoria. Si spieghi il perché e si proponga una possibile correzione al problema.
 - B) Perché la funzione ritorna `size`, qualora non ci sia errore? E' possibile che, in una realizzazione più completa della funzione, il valore ritornato sia maggiore o minore di `size`?
-

Domanda 5 (Sincronizzazione Multi-core e Semantica dei Semafori)

Contesto: Sia dato un sistema operativo OS161.

Si risponda alle seguenti domande:

- B) Si spieghi perché, in un contesto multi-core (sono presenti più CPU) non è possibile realizzare la mutua esclusione semplicemente disabilitando e riabilitando l'interrupt.
- B) Dato il codice delle funzioni di semaforo P e V:

```
void P(struct semaphore *sem) {
    spinlock_acquire(&sem->sem_lock);
    while (sem->sem_count == 0) {
        wchan_sleep(sem->sem_wchan, &sem->sem_lock);
    }
    sem->sem_count--;
    spinlock_release(&sem->sem_lock);
}

void V(struct semaphore *sem) {
    spinlock_acquire(&sem->sem_lock);
    sem->sem_count++;
    KASSERT(sem->sem_count > 0);
}
```

```
wchan_wakeone(sem->sem_wchan, &sem->sem_lock);  
spinlock_release(&sem->sem_lock);  
}
```

Rispondere ai quesiti:

1. A cosa serve lo spinlock (in entrambe le funzioni)?
2. Perché la `P` contiene un ciclo `while` anziché un `if`?
3. Perché la `wchan_sleep` riceve come parametro lo spinlock?
4. E' possibile che la chiamata alla `wchan_wakeone` svegli più di un thread in attesa?

B) Spiegare brevemente la differenza tra condition variable e `wait_channel`.